Infraestructura Tecnológica

Virtualización (KVM)

Objetivos

Comprender:

- Qué es un hipervisor.
- Diferenciar Tipo 1 y Tipo 2.
- Entender la arquitectura KVM.

Administrar:

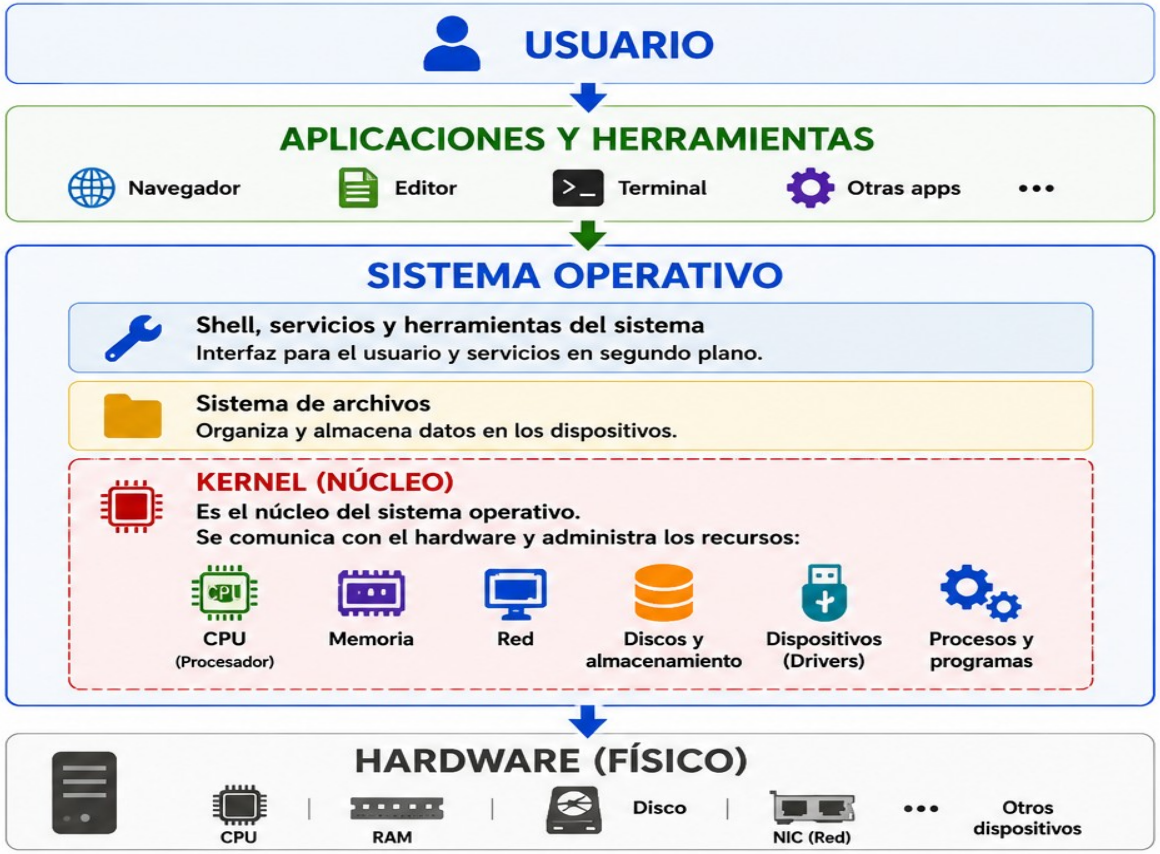
- Crear y administrar máquinas virtuales.
- Utilizar las principales herramientas (CLI y GUI).

Conocer:

- Almacenamiento virtual.
- Redes básicas (NAT y Bridge).
- Conceptos de troubleshooting.

¿QUÉ HAY DENTRO DE UN SISTEMA OPERATIVO?

Un sistema operativo es el intermediario entre las aplicaciones y el hardware.



SISTEMA OPERATIVO

Proporciona el entorno en el que se ejecutan las aplicaciones y administra los recursos de la computadora de forma segura y eficiente.

KERNEL (NÚCLEO)

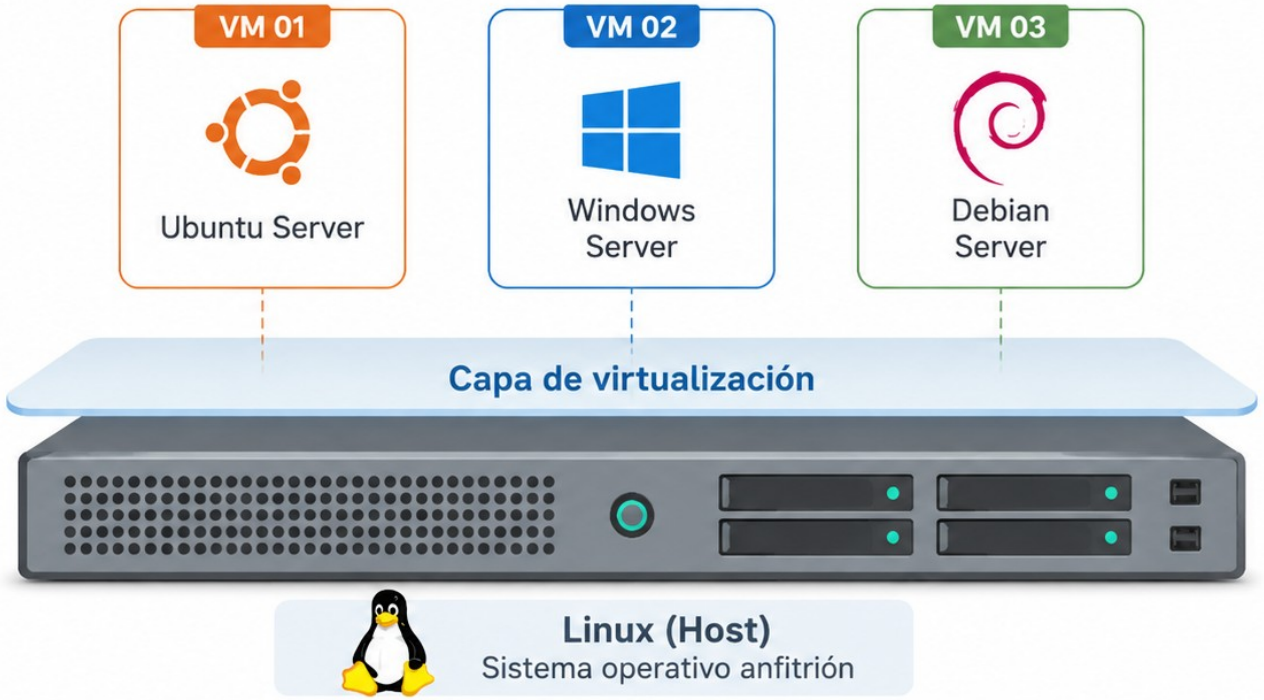
Es la parte central del sistema operativo. Tiene control total sobre el hardware y se encarga de administrar los recursos esenciales: CPU, memoria, dispositivos, almacenamiento, red y procesos.

El kernel es la capa central entre el software (aplicaciones) y el hardware.

Sin el sistema operativo (y su kernel), las aplicaciones no podrían acceder ni usar el hardware.

Virtualización con KVM/QEMU

Infraestructura virtual sobre Linux



Un servidor Linux permite ejecutar múltiples máquinas virtuales (VMs) independientes, cada una con su propio sistema operativo y aplicaciones.

¿Qué es virtualizar?

Virtualizar es crear una representación lógica de un recurso físico.

Cada máquina virtual se comporta como si fuera una computadoro física **independiente.**

IDEA CLAVE

Gracias a la virtualización, un único servidor físico puede ejecutar **múltiples máquinas virtuales independientes**, optimizando recursos, costos y administración.



AISLAMIENTO Y SEGURIDAD: Las VMs están aisladas entre sí y del host, garantizando estabilidad y seguridad.

¿Qué es un hipervisor?

Software (o capa del sistema) que permite crear y ejecutar (VM) máquinas virtuales compartiendo el hardware físico.

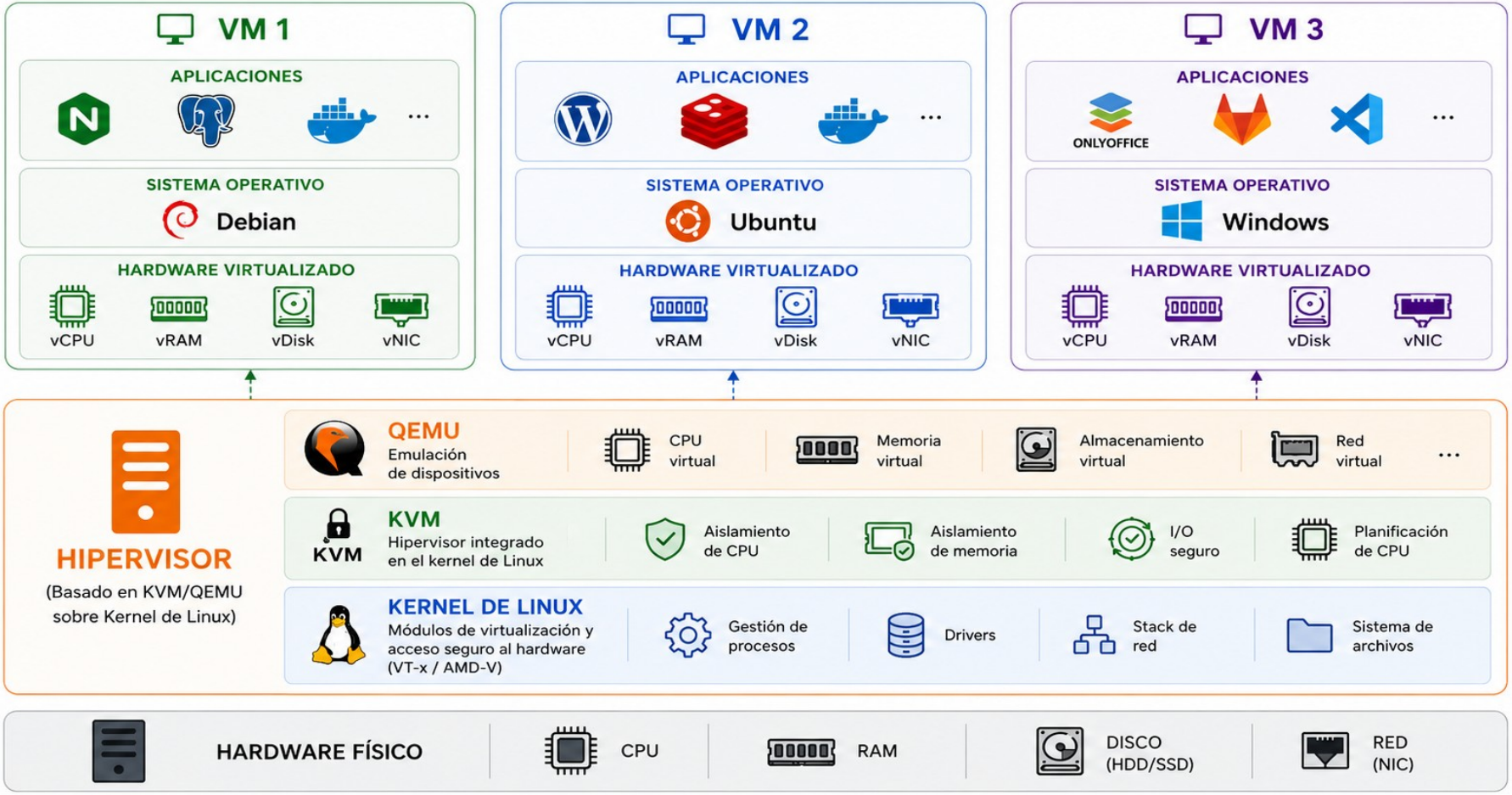
Funciones:

- asigna CPU
- asigna memoria
- aísla VMs
- administra acceso al hardware
- ejecuta múltiples sistemas operativos

La virtualización necesita una **capa que administre y aísla recursos compartidos** entre múltiples VMs: esa capa es el hipervisor.

Hipervisor: Crea y ejecuta VMs

EXACTAS!

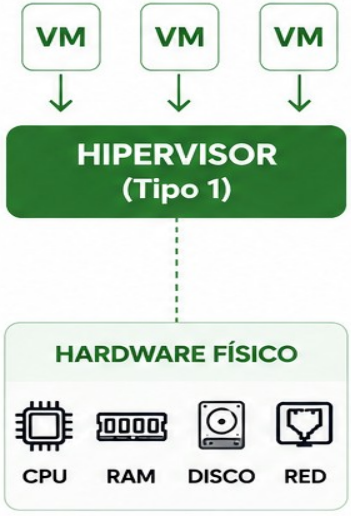


Hipervisor Tipo 1 vs Tipo 2

Dos formas de virtualizar

TIPO 1
(BARE-METAL)

Se ejecuta directamente sobre el hardware

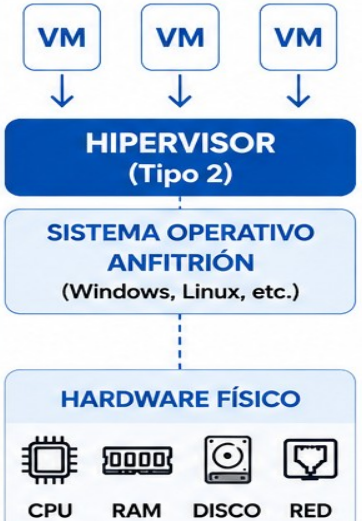


Acceso directo al hardware

TIPO 1 (BARE-METAL)	CARACTERÍSTICA	TIPO 2 (HOSTED)
Directamente sobre el hardware	Dónde se ejecuta	Dentro de un sistema operativo anfitrión (Windows, Linux, etc.)
Alto (acceso directo al hardware)	Rendimiento	Menor (intermediado por el sistema operativo)
Producción, servidores, datacenters	Uso principal	Laboratorios, desarrollo, entorno doméstico
Proxmox VE, Xen, KVM/Linux, VMware ESXi, Hyper-V Server	Ejemplos	VirtualBox, VMware Workstation, VMware Fusion, Parallels
Requiere administrar infraestructura dedicada	Implementación	Se instala como una aplicación más
Completo	Acceso al hardware	Limitado por el host
Sí (opcional)	Soporte de virtualización anidada	Depende del host y configuración
Servidores de producción, entornos escalables	Ideal para...	Clases, pruebas, simulaciones, portátiles

TIPO 2
(HOSTED)

Se ejecuta dentro de un sistema operativo anfitrión



Acceso al hardware a través del sistema operativo

VIRTUALIZACIÓN ASISTIDA POR HARDWARE

VT-x (Intel) y AMD-V (AMD) son extensiones del CPU que permiten ejecutar máquinas virtuales de forma **eficiente y segura** con soporte directo del hardware.

ANTES: sin VT-x / AMD-V

ANTES:
virtualizar en arquitecturas x86 era posible, pero requería técnicas complejas de virtualización por software con **mayor sobrecarga y menor rendimiento.**

La virtualización por software traduce/ emula cada instrucción, lo que requiere más recursos del host y **reduce el rendimiento.**



Mayor sobrecarga • Menor rendimiento

AHORA: con VT-x / AMD-V



Menor sobrecarga • Rendimiento cercano al nativo

AHORA:
con VT-x y AMD-V, el procesador incorpora un modo de ejecución específico para máquinas virtuales, permitiendo que el hipervisor ejecute múltiples sistemas operativos (guest) aislados de forma **más eficiente y segura.**

KVM
utiliza estas extensiones del CPU para ejecutar las máquinas virtuales con un **rendimiento cercano al nativo.**

EN RESUMEN: VT-x (Intel) y AMD-V (AMD) permiten que el hardware ayude al hipervisor a ejecutar máquinas virtuales de manera **más eficiente, segura y con mejor rendimiento.**

¿Qué es KVM?

KVM es una plataforma de virtualización integrada en el **kernel de Linux** para crear y administrar máquinas virtuales.

1 Plataforma de virtualización

KVM trabaja junto con otros componentes para ofrecer una plataforma completa de virtualización.

KVM
Virtualiza el CPU usando VT-x / AMD-V.

QEMU
Emula el hardware de la máquina.

libvirt
Administra la infraestructura de virtualización.

virsh
Interfaz de línea de comandos (CLI).

virt-manager
Interfaz gráfica para administrar la plataforma.

2 Crear y administrar máquinas virtuales

Permite crear, configurar, iniciar, detener y administrar máquinas virtuales de distintos sistemas operativos.

MÁQUINAS VIRTUALES (VMs)

Linux · Windows · Ubuntu · openSUSE · ...

3 Sobre hardware físico

Las máquinas virtuales se ejecutan sobre el hardware físico del servidor.

HARDWARE FÍSICO

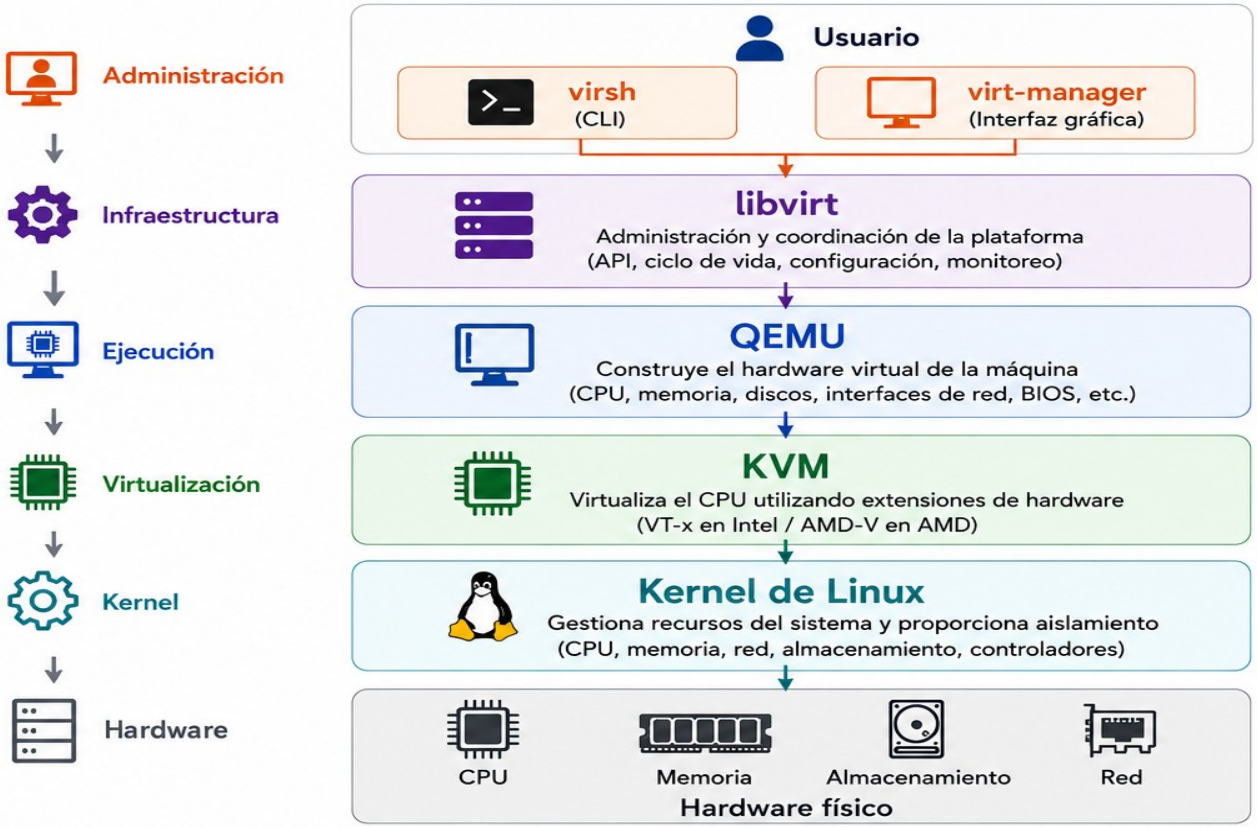
CPU · Memoria RAM · Almacenamiento · Red

4

KVM es una arquitectura de virtualización compuesta por varios componentes que trabajan en conjunto para brindar una solución eficiente, flexible y poderosa.

Arquitectura completa de KVM/QEMU

Cada componente cumple una función específica y trabaja junto con los demás para crear y ejecutar máquinas virtuales.



Operan la plataforma
Herramientas para crear, administrar y monitorear máquinas virtuales.

Administra y coordina
Proporciona una API unificada para gestionar VMs, redes, almacenamiento y permisos.

Construye el hardware virtual
Emula los dispositivos y brinda el entorno completo que la VM necesita para funcionar.

Ejecuta las máquinas virtuales
Aprovecha las extensiones del CPU para ejecutar las VMs con rendimiento cercano al nativo.

Gestiona los recursos del sistema
El kernel de Linux asigna y protege los recursos físicos entre las diferentes VMs.

Recursos físicos
El hardware real donde todo se ejecuta.

Las siguientes diapositivas analizan el rol de cada componente en detalle.

virsh (Interfaz de línea de comandos)

Herramienta CLI para administrar máquinas virtuales a través de libvirt.

```
$ virsh list
 Id    Nombre          Estado      CPU    Memoria
----------------------------------------------------
 1     debian12        running     2      2048 MiB
 2     ubuntu22        shut off    -      -
 3     centos9         running     1      1024 MiB
```

Consulta
Lista las máquinas virtuales y su estado.

```
$ virsh start ubuntu22
Dominio ubuntu22 iniciado

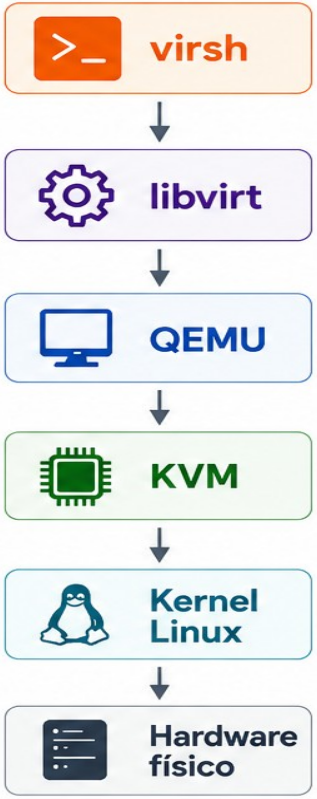
$ virsh shutdown ubuntu22
Dominio ubuntu22 apagado

$ virsh reboot debian12
Dominio debian12 reiniciado
```

Operaciones
- start (inicia)
- shutdown (apaga)
- reboot (reinicia)

```
$ virsh console debian12
Conectando al dominio debian12 ... (Presione Ctrl+] para salir)
debian12 login: 
```

Acceso
La consola permite acceder a la consola de la VM.

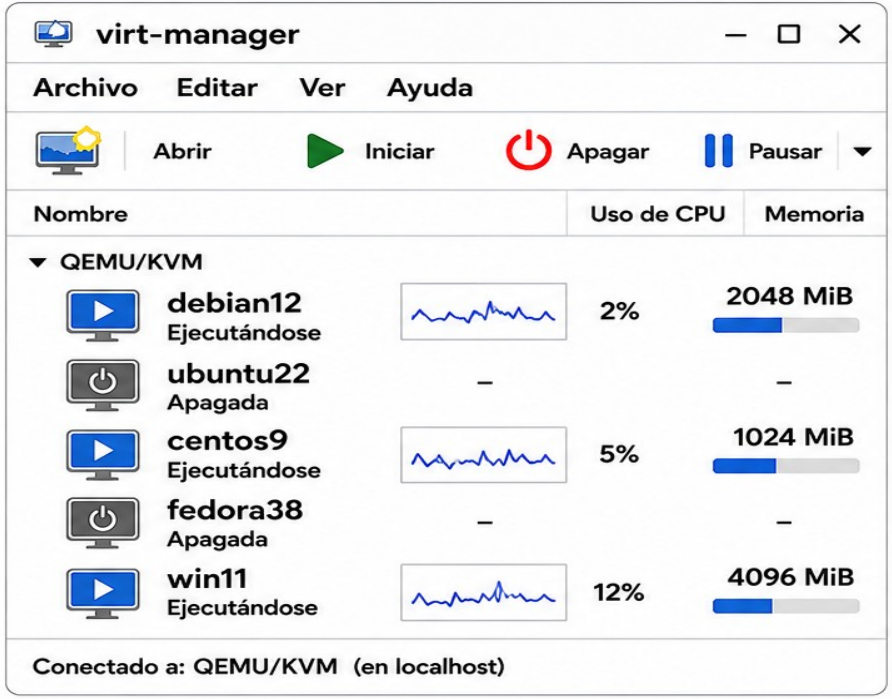


virsh no administra directamente las máquinas virtuales.
Todas las operaciones se realizan a través de **libvirt**.



virt-manager (Interfaz gráfica)

Interfaz gráfica para administrar máquinas virtuales a través de libvirt.



Vista general

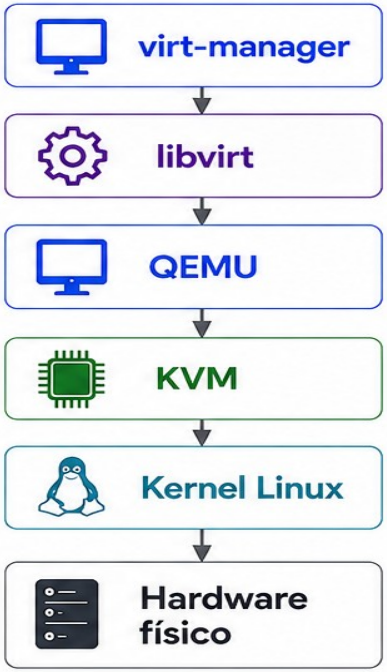
Muestra el estado de las máquinas virtuales: ejecutándose, apagadas, uso de CPU y memoria.

Gestión de VMs

- Crear nuevas máquinas virtuales
- Iniciar, apagar, pausar, reiniciar
- Asignar recursos (CPU, RAM, disco, red)
- Configurar dispositivos
- Tomar snapshots
- Consola de la máquina virtual

Consola integrada

Permite acceder a la consola gráfica o de texto de la máquina virtual.



virt-manager no administra directamente las máquinas virtuales.
Todas las operaciones se realizan a través de **libvirt**.



libvirt (Administración de la plataforma)

Proporciona una **API unificada** para gestionar todos los recursos de la virtualización.

Máquinas Virtuales

Crea, inicia, detiene, pausa y elimina VMs.

Redes

Define y administra redes virtuales, puentes y NAT.

Almacenamiento

Gestiona volúmenes, pools, imágenes ISO y discos virtuales.

Snapshots

Crea y administra instantáneas de las VMs.

libvirt

API de administración

(daemon: libvirtd)

Permisos y acceso

Controla usuarios, grupos y permisos sobre los recursos.

Monitoreo y estados

Proporciona información del estado de la plataforma y de las VMs.

libvirt expone una API estable y unificada (CLI, GUI y SDK) para gestionar todos los recursos de la plataforma de virtualización.

- virsh (CLI)
- libvirt (Administración)
- QEMU (Hardware virtual)
- KVM (Virtualización)
- Kernel Linux (Sistema)
- Hardware físico (Recursos)

libvirt (Coordina los componentes)

Recibe las órdenes de las herramientas y las traduce en acciones sobre los componentes inferiores.

Herramientas

- virsh (CLI)
- virt-manager (GUI)

Envían órdenes

libvirt

- Interpreta las órdenes
- Valida y aplica políticas
- Gestiona el ciclo de vida
- Coordina los recursos

API / Daemon (libvirtd)

Se comunica con

QEMU

Construye el hardware virtual y ejecuta la VM.

Con KVM

Usa KVM para aceleración por hardware.

Sin KVM

QEMU emula por software y usa el kernel de Linux.

Kernel Linux

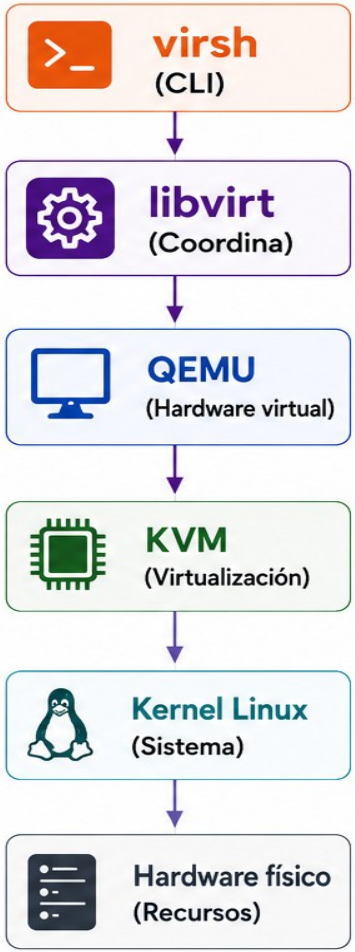
(Sistema operativo)

Hardware físico

(Recursos del servidor)

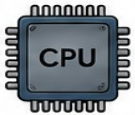
libvirt NO ejecuta las máquinas virtuales ni accede directamente al hardware.

Su trabajo es coordinar, administrar y abstraer todos los componentes de la plataforma.



QEMU

Emula el **hardware** y **construye** la computadora donde se ejecuta la máquina virtual.



CPU
Procesador



RAM
Memoria principal



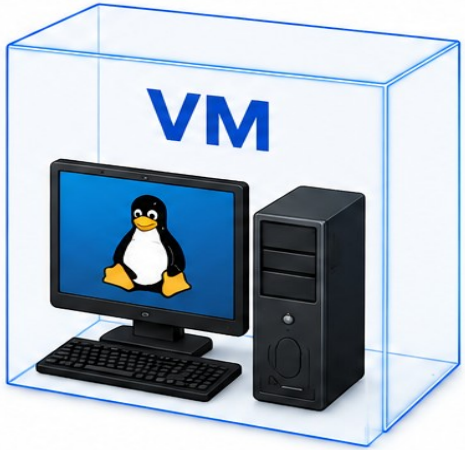
NIC
Tarjeta de red



Disco
Almacenamiento



BIOS
Firmware / Arranque



virsh
(CLI)

libvirt
(Coordina)

QEMU
(Hardware virtual)

KVM
(Virtualización)

Kernel Linux
(Sistema)

Hardware físico
(Recursos)

QEMU crea y ejecuta la máquina virtual emulando el hardware.
Cada máquina en ejecución corresponde a un **proceso QEMU** en el sistema operativo Linux.

QEMU

QEMU puede funcionar de dos maneras:

CON KVM

QEMU delega la ejecución del código del sistema operativo invitado al módulo KVM del kernel Linux, que trabaja directamente sobre el hardware físico (bare metal).

- **Sistema operativo invitado (Guest)**
- ↓
- **QEMU** (Hardware virtual)
- ↓
- **KVM** Módulo KVM del kernel Linux
- ↓
- **CPU** VT-x / AMD-V (aceleración por hardware)
- ↓
- **Rendimiento cercano al nativo**

SIN KVM

QEMU ejecuta el código del sistema operativo invitado mediante emulación por software.
Rendimiento considerablemente menor.

- **Sistema operativo invitado (Guest)**
- ↓
- **QEMU** (Hardware virtual)
- ↓
- **Kernel Linux** (Sistema)
- ↓
- **Hardware físico** (Recursos)

Arquitectura

- **virsh** (CLI)
- ↓
- **libvirt** (Coordina)
- ↓
- **QEMU** (Hardware virtual)
- ↓
- **KVM** (Virtualización)
- ↓
- **Kernel Linux** (Sistema)
- ↓
- **Hardware físico** (Recursos)

Una VM en **ejecución** = un **proceso QEMU** + recursos administrados por el kernel Linux.
El uso de **KVM** es opcional, pero proporciona un rendimiento muy superior.

KVM / Linux

KVM está integrado en el kernel Linux.

Linux

CPU	Memoria	Networking	Storage	Drivers	Seguridad
Scheduler Planificador de procesos	Gestión de memoria	Pila de red (TCP/IP)	Sistemas de archivos	Drivers de dispositivos	Permisos y aislamiento

KVM

Módulo del kernel Linux

A diferencia de otros hipervisores que implementan su propia plataforma de virtualización o un kernel propio, KVM está integrado en el kernel Linux.

Linux nativamente sabe administrar los recursos del sistema:

- CPU (planificador de procesos)
- Memoria
- Networking
- Storage
- Drivers de dispositivos
- Seguridad y permisos

virsh (CLI)

↓

libvirt (Coordina)

↓

QEMU (Hardware virtual)

↓

KVM (Virtualización)

↓

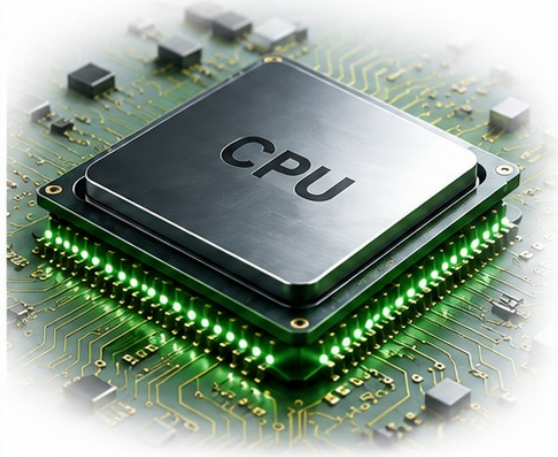
Kernel Linux (Sistema)

↓

Hardware físico (Recursos)

KVM

KVM usa las capacidades de virtualización del procesador.



VT-x (Intel)

AMD-V (AMD)

KVM (Kernel-based Virtual Machine)

Guest (Sistema operativo invitado)

KVM (Kernel-based Virtual Machine): módulo del kernel Linux que agrega capacidades de virtualización.

- Utiliza las extensiones VT-x (Intel) o AMD-V (AMD).
- Permite ejecutar múltiples máquinas virtuales de forma aislada.
- Ejecuta el código del sistema operativo invitado con acceso eficiente al procesador.
- Trabaja en conjunto con QEMU para ofrecer una plataforma completa de virtualización.

QEMU construye la computadora virtual; **KVM** permite ejecutarla aprovechando las capacidades de virtualización del procesador.

virsh (CLI)

↓

libvirt (Coordina)

↓

QEMU (Hardware virtual)

↓

KVM (Virtualización)

↓

Kernel Linux (Sistema)

↓

Hardware físico (Recursos)

¿CÓMO TRABAJAN JUNTOS?

KVM / QEMU: ARQUITECTURA COMPLETA



USUARIO — Interactúa con el sistema de virtualización

- virsh (CLI)
- virt-manager (GUI)

↓

LIBVIRT — Administra y coordina las máquinas virtuales

- API / Daemon (libvirtd)

↓

QEMU — Construye la máquina virtual (hardware virtual)

- vCPU
- vRAM
- vRed
- vDisco
- vBIOS

↓

KVM — Permite ejecutar el sistema operativo invitado usando la virtualización por hardware

- VT-x (Intel) o AMD-V (AMD) ....
- Guest OS
- Guest OS
- ...
- Guest OS

↓

KERNEL LINUX — Administra los recursos del sistema y proporciona aislamiento y seguridad

- Scheduler (CPU)
- Memoria
- Networking
- Storage
- Drivers
- Seguridad y permisos

↓

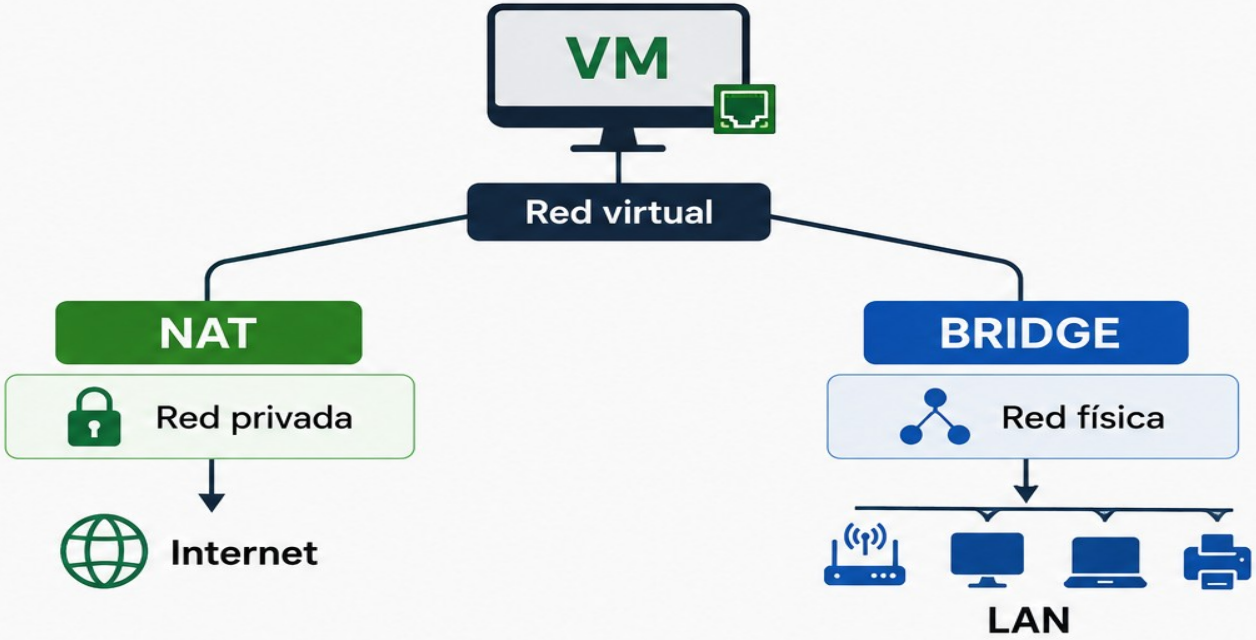
HARDWARE FÍSICO — Recursos reales del equipo

- CPU
- Memoria
- Red
- Discos
- Dispositivos

virsh / virt-manager operan → libvirt administra y coordina → QEMU construye la VM → KVM permite ejecutarla eficientemente → Linux administra los recursos del hardware.

Conectividad de las máquinas virtuales

Una máquina virtual necesita una **interfaz de red** para comunicarse con otras máquinas y con el exterior.



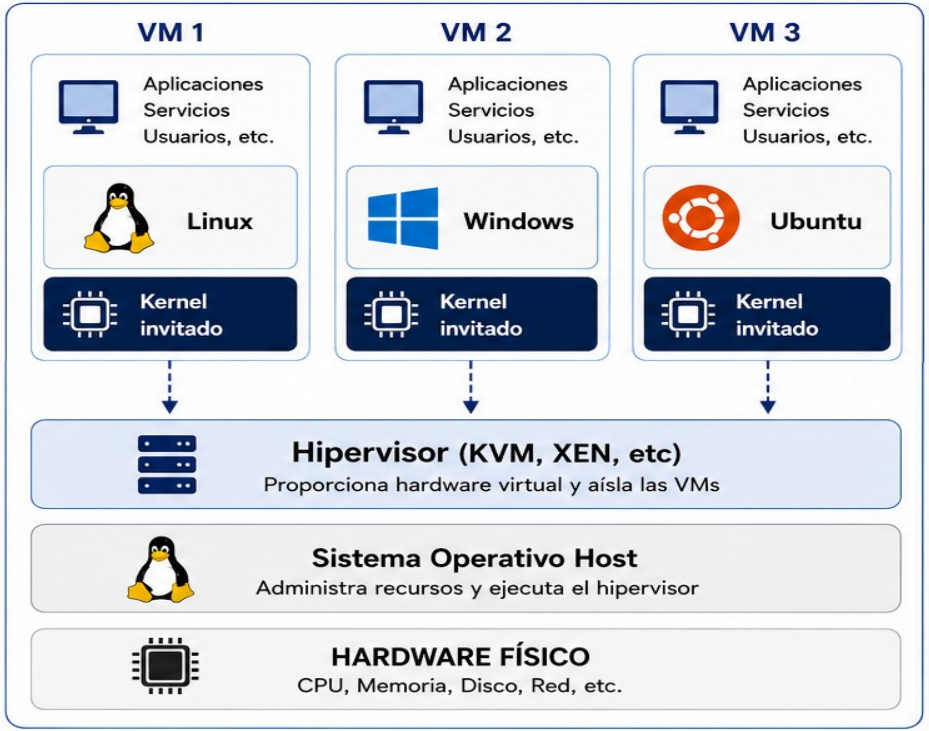
NAT (predeterminado): la VM utiliza una red privada administrada por libvirt y accede a Internet mediante NAT.

Bridge: la VM se conecta directamente a la red física y se comporta como un equipo más de la LAN.

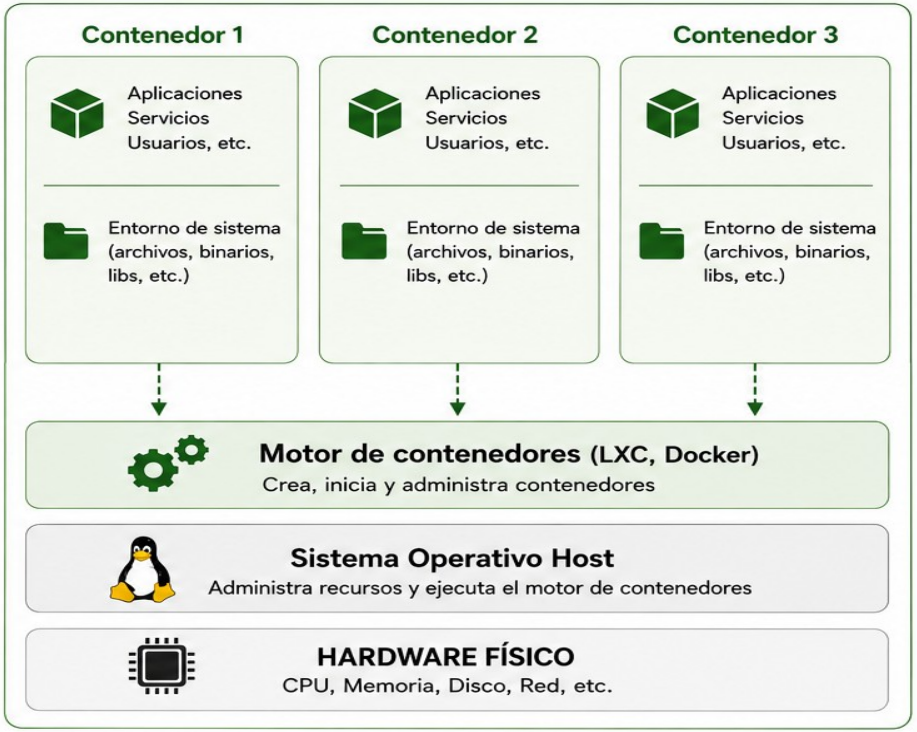
Más adelante veremos cómo implementar ambos modelos con KVM/libvirt.

MÁQUINAS VIRTUALES vs CONTENEDORES

Cada VM tiene su propio sistema operativo y kernel



Cada contenedor tiene su propio entorno aislado, pero comparte el kernel del host



Máquinas Virtuales: cada VM incluye un sistema operativo completo con su propio kernel. Pueden ejecutar distintos sistemas operativos (Linux, Windows, BSD, etc.) de forma aislada.

Contenedores: cada contenedor tiene su propio entorno aislado (archivos, procesos, usuarios, servicios, etc.), pero comparte el kernel del host. Son más ligeros y rápidos de iniciar.

ARQUITECTURA KVM:
COMPONENTES, FUNCIONES E INTERACCIÓN

La plataforma KVM se compone de capas que trabajan en conjunto
para crear, ejecutar y administrar máquinas virtuales de forma eficiente y segura.

VIRSH / VIRT-MANAGER	Envían comandos a libvirt	Herramientas con las que el usuario administra las máquinas virtuales.
LIBVIRT	Gestiona y coordina la infraestructura virtual	Daemon que recibe los comandos de las herramientas de administración y gestiona toda la infraestructura de virtualización.
QEMU	Construye el hardware virtual y ejecuta el proceso de la máquina virtual	Emula el hardware virtual (vCPU, vRAM, vDisco, vRed, vBIOS, etc.) y presenta los dispositivos al sistema operativo invitado.
KVM	Utiliza las extensiones de virtualización del procesador (VT-x / AMD-V) para ejecutar el guest	Permite ejecutar el código del sistema operativo invitado de forma eficiente utilizando el hardware de virtualización (VT-x / AMD-V).
KERNEL LINUX	Administra los recursos del sistema y proporciona aislamiento y seguridad	Gestiona CPU, memoria, red, almacenamiento, dispositivos y controladores.
HARDWARE FÍSICO	Proporciona los recursos físicos del servidor	CPU, memoria RAM, discos, interfaces de red, controladoras y otros dispositivos físicos.

La virtualización con **KVM** no depende de un único programa:
es el resultado de la colaboración entre herramientas de administración,
libvirt, **QEMU**, **KVM**, el **kernel Linux** y el **hardware**.